

MC809 Computação Distribuída

Laboratório 2

Nome: Arthur Costa Barreto

RA: 206325

Data: Setembro de 2026

1. Descrição da Proposta do Laboratório

O laboratório trata de uma simulação de comunicação entre clientes (bizantinos ou não) e um servidor. O objetivo é realizar a comunicação entre cliente e servidor utilizando gRPC, em python, e com base nos dados enviados pelos clientes identificar quais enviaram informações modificadas. Após essa identificação, utilizar apenas os dados enviados por clientes não bizantinos para treinar um modelo de machine learning e, ao final do treino com todos os dados, avaliar os resultados obtidos em um dataset de teste.

Para discutir os impactos de clientes bizantinos na comunicação com o servidor, 4 cenários diferentes são impostos, mas todos com 5 clientes ao total: apenas 1 cliente bizantino, 2 clientes bizantinos, 3 clientes bizantinos ou nenhum cliente bizantino. Em cada um dos casos, os clientes podem enviar dados alterados de forma randômica, com labels flipadas (0 vira 1 e vice-versa com probabilidade de 50%), ou com ruído adicionado em apenas uma feature.

2. Solução implementada

2.1 Arquivo Proto

Para este projeto, utilizou-se apenas um serviço denominado ByzantineManager, que recebe uma mensagem “DataRequest” e retorna uma mensagem “DataResponse”. DataRequest possui as features: client_id (int32), behavior (string), labels (repeated int32) e features (repeated FeatureRow, mensagem que possui repeated double e armazena as features de uma única amostra). Já DataResponse possui as features: isBizantine (bool) e message (string).

Como o objetivo é apenas simular um comportamento, o comportamento do cliente é enviado para o servidor pela feature “behavior”, mas é utilizada apenas para acrescentar informações nos logs exibidos no terminal.

2.2 Arquivo baseline_data

Para a simulação, um dataset extenso (N_SAMPLES=20.000) com 4 features e apenas 2 classes é gerado pela função *make_classification* do scikit-learn. Destas 20.000 amostras, 10.000 são reservadas para a avaliação do modelo final e 10.000 são usadas durante a execução do experimento. Dentre as 10.000 amostras de experimento, 5.000 são utilizadas por um modelo inicial que fica no servidor e é responsável por identificar dados de clientes

bizantinos, enquanto o restante é distribuído igualmente entre os clientes (1.000 amostras para cada cliente).

2.3 Cliente

Além do que já foi mencionado sobre a divisão dos dados entre os clientes, vale mencionar que a classe implementada em python, `Client`, é marcada com o anotador `ray.remote`. Dessa forma, os dados são enviados de forma paralela durante a execução de cada um dos 4 cenários, como exigido no laboratório.

Outro ponto relevante é que para criar um objeto `Client` deve-se especificar o comportamento (*behavior*) e passar os dados que serão enviados para o servidor por aquele cliente, de forma que já na criação do objeto os dados são alterados ou não de acordo com o *behavior*.

2.4 Servidor

A classe do servidor possui como único atributo um objeto definido no mesmo arquivo, que é um `ray.remote` chamado de `Aggregator`. Isso é feito para que os dados enviados de forma paralela pelos clientes sejam armazenados e o processo de classificação/treino seja feito apenas quando todos os clientes finalizarem o envio de dados, assim como é pedido na definição do laboratório.

Ademais, vale comentar que dentre as 5.000 amostras reservadas para o servidor, 2.500 são utilizadas para treinar um `SGDClassifier` (classe do scikit-learn para um classificador por gradiente descendente estocástico) e 2.500 são utilizadas para avaliação. Dessa forma, pode-se ter uma noção de como o modelo se comporta com dados desconhecidos e, a partir de experimentação, definiu-se como regra de identificação de clientes bizantinos a seguinte equação:

$$Threshold = 0.45 * (score_{eval} + score_{train})$$

onde $score_{eval}$ representa a acurácia das 2.500 amostras de avaliação e $score_{train}$ representa a acurácia das 2.500 amostras de treino.

Note que o threshold é apertado em relação ao score de treino, mas isso é feito dessa forma porque parte dos clientes bizantinos estão alterando apenas uma das 4 features, o que torna mais difícil a identificação.

A classe `SGDClassifier` do scikit learn permite o treinamento incremental do modelo de classificação, então após os clientes enviarem os dados e passarem pela identificação de clientes bizantinos do servidor, um treinamento incremental é feito em cima dos dados de clientes honestos. Por fim, após o treinamento incremental com os dados extras, julga-se a melhora/piora do modelo base (todos os clientes honestos) em acurácia para classificação das 10.000 amostras separadas no começo do experimento para teste.

3. Resultados

O experimento foi repetido 10 vezes para cada cenário, isto é, 10 execuções com 5 clientes honestos, 10 execuções com 4 clientes honestos, e assim por diante. Dentre cada execução, tanto o valor de cada amostra original quanto o split de dados manteve-se o mesmo, garantidos ao setar *random_state*=42. No entanto, o que de fato variou foi a aplicação randômica de ruído aos dados de clientes bizantinos, já que cada uma das distorções citadas no tópico 1 é aplicada sob um regime aleatório sem *random_state* especificado.

A seguir, exibe-se o resultado do experimento para a configuração ótima (fator de 0.45 no threshold) e subótima (fator de 0.4 no threshold). Vale ressaltar que a % de erro é sobre um total de 50 clientes, pois cada uma das 10 execuções do experimento acontece com 5 clientes; e que erro é definido quando a regra de classificação julga um cliente como honesto quando na verdade ele é bizantino.

Métrica	0 Bizantinos	1 Bizantino	2 Bizantinos	3 Bizantinos
Acurácia média	0,6698	0,6882	0,6708	0,6704
Acurácia mínima	0,4887	0,6389	0,5584	0,6134
Acurácia máxima	0,7222	0,7136	0,7224	0,7077
Desvio Padrão	0,0724	0,0239	0,0544	0,0317
% erro de classificação	0%	0%	0%	0%

Tabela 1: Threshold ótimo sob fator de 0.45

Métrica	0 Bizantinos	1 Bizantino	2 Bizantinos	3 Bizantinos
Acurácia média	0,6732	0,6525	0,6214	0,5861
Acurácia mínima	0,5592	0,5785	0,544	0,3411
Acurácia máxima	0,7039	0,7061	0,6789	0,7135
Desvio Padrão	0,0453	0,0412	0,0425	0,1101
% erro de classificação (E/50)	0%	20%	18%	36%

Tabela 2: Threshold subótimo sob fator de 0.40

4. Conclusão

A partir dos resultados demonstrados na seção anterior, verifica-se que a regra de decisão ótima se mostrou muito eficiente, errando na classificação de 0 clientes. Por essa razão, os resultados finais de acurácia nos dados de teste pouco variam em cada cenário, sendo possível notar que, na média, a acurácia nos dados de teste praticamente não foi alterada pelo fato de o modelo do servidor ter menos dados de treino. Isso porque cada modelo incrementa apenas em 1.000 amostras o que o modelo base já conhece. Lembre que ele foi treinado com 2.500 amostras, então no pior cenário o modelo treina com 4.500 amostras ao invés de 7.500.

No entanto, quando o fator de decisão se torna mais abrangente e clientes bizantinos passam despercebidos, o resultado muda completamente. Agora o cenário de apenas clientes honestos apresenta vantagem clara sobre os outros. Veja que a acurácia média obtida no cenário de 3 clientes bizantinos cai de 0,6704 para 0,5861 - redução de 13% na acurácia -, podendo chegar no pior caso em que a acurácia mínima obtida nas 10 execuções cai de 0,6134 para 0,3411 - redução de 45%. Essa piora expressiva é causada pela distorção no modelo base que agora é treinado com dados alterados no cliente para prejudicar propositalmente a classificação do modelo de machine learning.

5. Observação

É possível reproduzir as 10 execuções consecutivas a partir do arquivo *run_experiment.py*, que foi gerado com IA. No entanto, vale ressaltar que os resultados não necessariamente serão iguais aos reportados já que a alteração sobre os dados de clientes bizantinos é randômica.